



Furniture (Solution)

We may regard the pieces of furnitures initially located in the room are located by operations. It is enough to consider the solution if there is no furniture in the room in the beginning. More concretely, for each i, j satisfying $C_{i,j} = 1$, we add a new operation to the block (i, j) in the beginning. We do not output the results of the new operations.

Subtask 1

In this subtask, it is possible to perform $O(NM)$ calculations for each operation. Therefore, for each operation, we temporarily locate a piece of furniture at the block, and decide whether the arrangement is nice or not. If it is not nice, we remove the piece of furniture.

By breadth first search or the depth first search, it is possible to decide whether the arrangement is nice or not in $O(NM)$ time.

Subtask 2 (Full Score)

A move satisfying the condition of a nice arrangement will be called a **nice move**. If there is a nice move passing through the block (i, j) , we put $R_{i,j} = 0$. Otherwise, we put $R_{i,j} = 1$.

Assume that we perform an operation to the block (X, Y) .

1. Case: $R_{X,Y} = 1$.

In this case, there is no move passing through the block (X, Y) . If we locate a piece of furniture at the block (X, Y) , a nice move will remain nice. Therefore, the arrangement is nice.

2. Case: $R_{X,Y} = 0$.

If we locate a piece of furniture at the block (X, Y) , a nice move passing through (X, Y) will not remain nice. We need to consider whether there exist other nice moves.

- (a) Assume that there exists (V, W) satisfying $R_{V,W} = 0$, $V + W = X + Y$, and $(V, W) \neq (X, Y)$.

In this case, the arrangement will remain nice if we locate a piece of furniture at (X, Y) . In fact, there exists a nice move passing through (V, W) . Since such a move does not pass through (X, Y) , it remains nice if we locate a piece of furniture at (X, Y) . For a nice move, the value of $x + y$ for a block (x, y) which we pass through will increase. Hence we do not pass two blocks with the same value of $x + y$.

- (b) Assume that such (V, W) does not exist.



In this case, the arrangement will not remain nice if we locate a piece of furniture at (X, Y) because every nice move will pass through (X, Y) . There will be no nice move if we locate a piece of furniture at (X, Y) . which we pass through will be increase by 1. Hence it must pass through (v, w) satisfying $X + Y = v + w$. There is no such (v, w) other than (X, Y) in this case.

Therefore, it is enough to manage the values of $R_{i,j}$ efficiently. If we locate a piece of furniture at (X, Y) , we update as $R_{X,Y} \leftarrow 1$. It may happen that we need to update as $R_{x,y} \leftarrow 1$ for some other (x, y) . Such (x, y) are detected as follows.

- Case: $R_{x+1,y} = 1$ and $R_{x,y+1} = 1$.

In this case, we cannot travel from (x, y) to (N, M) . Hence there does not exist a nice move passing through (x, y) .

- Case $R_{x-1,y} = 1$ and $R_{x,y-1} = 1$.

In this case, we cannot travel from $(1, 1)$ to (x, y) . Hence there does not exist a nice move passing through (x, y) .

Conversely, if there is no block satisfying the above conditions, the values of R are updated correctly. In fact, for any (x, y) with $R_{x,y} = 0$, either $R_{x+1,y} = 0$ or $R_{x,y+1} = 0$ is satisfied. Hence we may move to such block. Continuing this process, we will arrive at (N, M) . Similarly, repeating to move from (x, y) to $(x - 1, y)$ or $(x, y - 1)$, we will arrive at $(1, 1)$. Connecting two paths, we get a nice move.

Therefore, for each update $R_{x,y} \leftarrow 1$, we need to find adjacent blocks where the value of R should be updated. Once we update as $R_{x,y} = 1$, then the value of $R_{x,y}$ will never be 0 again. Therefore, the time complexity of this algorithm is $O(Q + NM)$.



Monochrome Points (Solution)

Subtask 1

Under the constraints of Subtask 1, we can try all $N!$ possibilities ways to draw line segments. Since we can calculate the intersection number in $O(N^2)$ time, this subtask can be solved in $O(N^2 \cdot N!)$ time.

Subtasks 2,3

Let $w_1, w_2, \dots, w_N$ be the positions of the white points in ascending order. Similarly, let $b_1, b_2, \dots, b_N$ be the positions of the black points in ascending order. Then, we shall prove that the best way to draw line segments is to connect w_i with b_{i+k} for every i for some integer k . Here i is an integer from 1 to N , and all indices are considered mod N .

Consider the best way to draw line segments, and assume that w_i is connected with b_{p_i} by a line segment. First, we shall prove that, for different i and j , the four points $w_i, w_j, b_{p_j}, b_{p_i}$ are not located on the circle in this order (in the clockwise order). In fact, if $w_i, w_j, b_{p_j}, b_{p_i}$ are located on the circle in this order, the intersection number will increase if we connect w_i with b_{p_j} , and w_j with b_{p_i} . It contradicts the assumption that we are considering the best way to draw line segments. Next, we shall show $p_i - i \equiv p_{i+1} - (i + 1)$ for every i . If it is not satisfied for some i , then there exists another black point b_{p_j} between b_{p_i} and $b_{p_{i+1}}$. Now we consider the positions of the four points $w_i, w_{i+1}, b_{p_i}, b_{p_{i+1}}$. By the above fact, $w_i, w_{i+1}, b_{p_{i+1}}, b_{p_i}$ are not located in this order. Similarly, $w_i, b_{p_i}, b_{p_{i+1}}, w_{i+1}$ are not located in this order. Assume that $w_i, w_{i+1}, b_{p_i}, b_{p_{i+1}}$ are located in this order. Since there does not exist any white point between w_i and w_{i+1} , the point w_j must lie between w_{i+1} and b_{p_j} , or between b_{p_j} and w_i . It contradicts the above fact. There are three remaining cases to consider, i.e., the case where $w_i, b_{p_{i+1}}, b_{p_i}, w_{i+1}$ are located in this order, the case where $w_i, b_{p_i}, w_{i+1}, b_{p_{i+1}}$ are located in this order, and the case where $w_i, b_{p_{i+1}}, w_{i+1}, b_{p_i}$ are located in this order. Similarly, we can derive a contradiction in every case.

Therefore, the best way to draw line segments is to connect w_i with b_{i+k} , for some integer k . There are N candidates of the value of k . For each k , if we calculate the intersection number in $O(N^2)$ time, we can solve the task in $O(N^3)$ time in total. If we use data structure such as the Binary Indexed Tree or the Segment Tree, we can calculate the intersection number in $O(N \log N)$ time in the same way as calculating the number of transpositions of a permutation. The total time complexity is $O(N^2 \log N)$, and we can solve Subtask 3.



Subtask 4 (Full score solution)

Similarly as in Subtasks 2, 3, we fix an integer k and connect w_i with b_{i+k} . We shall calculate the intersection number. For an index i , we consider the number of line segments which intersect with the line segment connecting w_i with b_{i+k} . We consider the two arcs whose endpoints are w_i, b_{i+k} . Let L, R ($L \leq R$) be the number of points on the arcs except for the endpoints. The number of line segments that intersect with the line segment connecting w_i with b_{i+k} is less than or equal to L . We shall prove that the equality actually holds, i.e. L line segments intersect with the the line segment connecting w_i with b_{i+k} .

Take one of the two arcs such that the number of points in the fixed arc is smaller than or equal to the number of points in the other arc. We also take a point in the fixed arc. We may assume without loss of generality that we take a white point. We denote it by w_j . Then, it is enough to prove that the four points $w_i, w_j, b_{j+k}, b_{i+k}$ are not located in this order, in the clockwise order and in the counterclockwise order. We assume that these four points are located in this order in the clockwise order, and we shall derive a contradiction. We can treat the case of the counterclockwise order similarly.

By assumption, there are L points on the arc from w_i to b_{i+k} . Since $L + R = 2N - 2$ and $L \leq R$, we have $L \leq N - 1$, i.e., the number of points on the arc is less than or equal to $N - 1$. Changing the labels of the white points if necessary, we may assume that $i < j$ holds. Then, the number of white points on the arc from w_i to w_j is $j - i - 1$, and the number of black points on the arc from b_{j+k} to b_{i+k} is $N - (j - i + 1)$. From this, we see that there are at least $(j - i - 1) + (N - (j - i + 1)) + 2 = N$ points on the arc from w_i to b_{i+k} , which is absurd.

Therefore, the number of line segments which intersect with the line segment connecting w_i with b_{i+k} is equal to the smaller number of points on the arcs between w_i and b_{i+k} except for the endpoints. (If the two arcs have the same number of points, we take any one of the two arcs.) By an appropriate case-by-case analysis, we can calculate this number by a linear function in terms of w_i and b_{i+k} . Thus we can solve this task if we can add a linear function on an interval of k for each i . This operation can be done in linear time by a cumulative sum technique. We need to find the interval where we will add a linear function. We can calculate it in linear time if we consider i in ascending order. In total, we can solve this task in $O(N)$ time.

Digression

Let $f(k)$ be the intersection number for each k . The function $f(k)$ satisfies certain monotonicity condition. Precisely, if we consider the function f is defined on the integers mod N , the following condition is satisfied.

Let a be the point where f takes the minimum, and b the point where f takes the maximum. Then, f is a monotone non-decreasing function in the interval $a, a + 1, \dots, b$, and it is a monotone non-increasing function in the interval $b, b + 1, \dots, a$.



To simplify the proof, we assume that the $2N$ points divide the circle equally and the length of the circumference is $2N$. For any two points p, q on the circle, let $d(p, q)$ be the length of the shorter arc whose endpoints are p, q . Let $d'(p, q)$ be the distance from p to q in the clockwise direction. Let $[p, q)$ be the arc from p to q in the clockwise direction. (Here the arc $[p, q)$ contains p but does not contain q .) If N black points are contiguous and N white points are contiguous, we can easily check the monotonicity condition. If this is not the case, the condition $d'(b_i, b_{i+1}) \leq N$ is satisfied for every i . In the following, we may assume this condition is satisfied.

We shall prove that the function $k \mapsto \sum_{i=1}^N d(w_i, b_{i+k})/2$ satisfies the monotonicity condition imposed on f . We put $g(k) = \sum_{i=1}^N (d(w_i, b_{i+k+1}) - d(w_i, b_{i+k}))/2$. We need to prove that both $\{k \mid g(k) \geq 0\}$ and $\{k \mid g(k) \leq 0\}$ are intervals. It is enough to prove that g satisfies the same monotonicity condition imposed on f . We shall prove this.

We take a point w on the circle and move it. We will consider the variation of the value of $(d(w, b_{i+k+1}) - d(w, b_{i+k}))/2$. Let b'_{i+k}, b'_{i+k+1} be the antipodal points to b_{i+k}, b_{i+k+1} , respectively. If we move w to the clockwise direction by a small quantity Δw , the variation is $-\Delta w$ if w is in the interval $[b_{i+k}, b_{i+k+1})$, the variation is 0 if w is in the interval $[b_{i+k+1}, b'_{i+k})$, the variation is Δw if w is in the interval $[b'_{i+k}, b'_{i+k+1})$, the variation is 0 if w is in the interval $[b'_{i+k+1}, b_{i+k})$. Hence $h(k) = g(k+1) - g(k)$ is given by the following formula:

$$h(k) = \sum_{i=1}^N \left| [w_{i-1}, w_i) \cap [b_{i+k}, b_{i+k+1}) \right| - \left| [w_{i-1}, w_i) \cap [b'_{i+k}, b'_{i+k+1}) \right|.$$

Here $|\cdot|$ denotes the length of an arc. For each integer p satisfying $1 \leq p \leq 2N$, we define s_p so that b_{s_p} is the black point closest to the p -th point (including the p -th point itself) in the counterclockwise direction. Similarly, we define t_p so that w_{t_p} is the white point closest to the p -th point (including the p -th point itself) in the counterclockwise direction. If s_p or t_p is equal to N , we replace its value by 0. For an integer p satisfying $p > 2N$, we define s_p, t_p by $s_p = s_{p-2N} + N, t_p = t_{p-2N} + N$. Then the above formula for $h(k)$ gives

$$\begin{aligned} h(k) &= \sum_{i=1}^N \left| \left\{ 1 \leq p \leq 2N \mid s_p \equiv i+k, t_p \equiv i-1 \right\} \right| - \left| \left\{ 1 \leq p \leq 2N \mid s_{p+N} \equiv i+k, t_p \equiv i-1 \right\} \right| \\ &= \left| \left\{ 1 \leq p \leq 2N \mid s_p - t_p \equiv k+1 \right\} \right| - \left| \left\{ 1 \leq p \leq 2N \mid s_{p+N} - t_p \equiv k+1 \right\} \right| \end{aligned}$$

Here the congruences are modulo N . This implies the function $g(k)$ is obtained by adding the constant 1 on $[s_p - t_p, s_{p+N} - t_p)$ and a constant function for each p . When we show the monotonicity condition, we can ignore differences of constant functions. Therefore, it is enough to prove that the function

$$k \mapsto \left| \left\{ 1 \leq p \leq 2N \mid k \in [s_p - t_p, s_{p+N} - t_p) \right\} \right|$$

satisfies the monotonicity condition imposed on f .



Arranging the labels of the points, we may assume that, for every i , the number of black points in the interval from the first point to the i -th point is greater than or equal to the number of white points in the interval from the first point to the i -th point. We can achieve it in the following way. We consider the number of black points in the interval from the first point to the i -th point minus the number of white points in the same interval. Take i so that this number takes the minimum. Then change the labels so that the i -th point becomes the first point in the new labeling.

By definition, the values of s_p, t_p are the number of black and white points in the interval from the first point to the p -th point, respectively. By the way to label the points, the inequality $s_p - t_p \geq 0$ holds for every p . Moreover, for every $1 \leq p \leq N$, since $s_{p+N} \leq N$, the inequality $s_{p+N} - t_p \leq N$ holds. Since $p \leq t_{p+N}$, we have $s_{p+2N} - t_{p+N} = N + s_p - t_{p+N} \leq N + s_p - p \leq N$. Therefore, for every $1 \leq p \leq 2N$, we have $s_{p+N} - t_p \leq N$. Hence, for every $1 \leq p \leq 2N$, we have $0 \leq s_p - t_p \leq s_{p+N} - t_p \leq N$. To prove the monotonicity condition, it is enough to prove that, for every $1 \leq p, q \leq 2N$, the inequality $s_q - t_q \leq s_{p+N} - t_p$ holds.

For p, q satisfying $p \leq q \leq p + N$, it follows from $s_q - s_{p+N} \leq 0 \leq t_q - t_p$. For p, q satisfying $p + N < q$, it follows from $s_q - s_{p+N} \leq q - (p + N) \leq t_q - t_p$. For p, q satisfying $q < p \leq N$, it follows from $s_q + t_p \leq p \leq s_{p+N} \leq s_{p+N} + t_q$. For p, q satisfying $q < p$ and $p > N$, if $p - q \leq N$ is satisfied, it follows from $s_{p+N} - s_q \geq (p + N) - q - N = p - q \geq t_p - t_q$. Finally, for p, q satisfying $q < p$ and $p - q > N$, if we put $r = p - N > q$, then we have $s_{p+N} - t_p = s_r + (N - t_p) \geq s_r \geq s_q \geq s_q - t_q$.

Therefore, for every $1 \leq p, q \leq 2N$, the inequality $s_q - t_q \leq s_{p+N} - t_p$ holds. and the assertion is proved.



Power Plant (Solution)

We shall interpret this task in terms of graph theory. The structure of the power plant becomes a tree, and the bases of the power plant become the vertices of the tree.

Subtask 1

Since the tree has a small number of vertices, we can try all possibilities of the status (ON/OFF) of the power generators' switches. Choose a vertex of a power generator whose status is ON, and consider a rooted tree whose root is the chosen vertex. By the depth-first search (DFS), we can calculate the profit in $O(N)$ time. Therefore, the total time complexity of this algorithm is $O(N2^N)$.

Since $N \leq 16$, we may calculate the profit using a slower algorithm which takes $O(N^2)$ or $O(N^3)$ time.

Subtask 1 (Another solution)

We consider the minimum subtree which contains all the power generators which are finally activated or broken. In such a subtree, all the leaves are activated power generators, and all power generators which are not leaves are broken. Therefore, considering all subtrees for which all leaves are power generators, we can solve this task by calculating the maximum of

$$(\text{The number of leaves}) - (\text{The number of power generators which are not leaves}).$$

Subtask 2

We fix a power generator whose status will be ON, and consider the problem of maximizing the profit when we turn the switch of the fixed power generator to ON. We consider the rooted tree whose root is the vertex of the fixed power generator. Then, a power generator other than the root is broken if and only if its descendants contain a power generator whose switch is ON. We may think that this task can be solved by dynamic programming.

For each vertex v , consider the subtree consisting of v and its descendants, and let dp_v be the maximum of

$$(\text{The number of activated power generators}) - (\text{The number of broken power generators}).$$

in the subtree when we choose the status (ON/OFF) of every power generator in the subtree. We calculate the value of dp_v by updating the values from the leaves successively.



Let us consider how to update the value of dp_v . For each vertex v , let E_v be the number of power generators at v ($E_v = 0, 1$). As explained above, when we determine the status of v , we consider the case where the descendants contain a power generator whose status is ON and the case where the descendants do not contain such a power generator, separately.

- (The case where the descendants of v do not contain a power generator whose status is ON)
In this case, the status of every descendant of v is OFF. If there is a power generator at v , we should turn its switch to ON. Therefore, the maximum value of dp_v is E_v .
- (The case where the descendants of v may contain a power generator whose status is ON)
Since we can freely choose the status (ON/OFF) of the descendants of v , we will calculate the sum of dp 's of the descendants of the children of v . But, if there is a power generator at v , it will be broken.

Summarizing these two cases, we have the following recurrence formula:

$$dp_v = \max \left\{ E_v, \left(\sum_{c: \text{a child of } v} dp_c \right) - E_v \right\}.$$

We shall calculate the values of dp_v by the depth-first search (DFS). Note that, at the root vertex, even if one child or the descendants of one child contain a power generator whose status is ON, the power generator at the root vertex will not be broken.

By the above algorithm, when we fix a power generator whose status will be ON, we can calculate the maximum of the profit in $O(N)$ time. Therefore, considering all choices of a fixed power generator, the original problem will be solved in $O(N^2)$ time.

Subtask 3 (Full score solution)

In the algorithm of Solution 2, we consider the rooted tree whose root is the fixed power generator whose status will be ON. We can calculate the value of dp at a vertex since we can decide whether the power generator at the vertex is broken or not using the information of its descendants only. In the following full score solution, we will also calculate the values of dp by considering a rooted tree. For each vertex, assuming there is a power generator whose status is ON outside the subtree consisting of its descendants, we will calculate the maximum of the profit when we choose the status (ON/OFF) of the subtree.

In order to calculate the maximum value completely, we consider the minimum subtree which contains all the power generators which are finally activated or broken. For each vertex v , we shall calculate the maximum value for the minimum subtree where the vertex closest to the root is v . If we can calculate this value for every vertex, we can solve the task.

Let us consider the calculate for the vertex v .



- (The case where there is a power generator at the vertex v , and it will be activated)

In this case, we can freely choose the status (ON/OFF) of a vertex in the subtree of one child only. The status of every vertex in the subtrees of other children should be OFF. Therefore, we shall calculate

$$\max \{ dp_c \mid c : \text{a child of } v \} + 1.$$

- (The case where there is a power generator at the vertex v , and it may be broken)

In this case, we can freely choose the status (ON/OFF) of a vertex of the descendants. We will calculate the sum of dp of the children of v . Assuming the power generator at v will be broken, we calculate

$$\left(\sum_{c : \text{a child of } v} dp_c \right) - 1.$$

- (The case where there is no power generator at the vertex v)

Similarly, since we can freely choose the status (ON/OFF) of a vertex of the descendants, we calculate

$$\sum_{c : \text{a child of } v} dp_c.$$

Summarizing, for each vertex v ,

- if $E_v = 1$, we calculate

$$\max \left\{ \max \{ dp_c \mid c : \text{a child of } v \} + 1, \left(\sum_{c : \text{a child of } v} dp_c \right) - 1 \right\}, \text{ and}$$

- if $E_v = 0$, we calculate

$$\sum_{c : \text{a child of } v} dp_c.$$

Then the maximum of these values will be the answer.

By the above algorithm, the answer can be calculated in $O(N)$ time.